

multiclass__binary

June 7, 2026

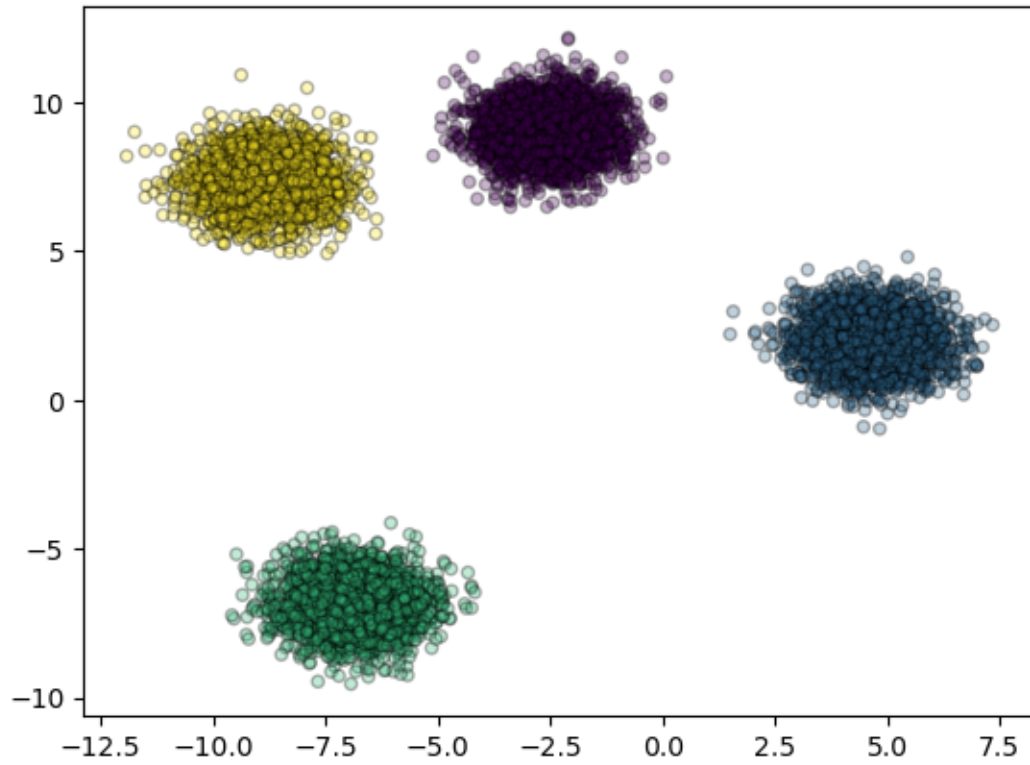
```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsOneClassifier
from sklearn.metrics import classification_report
import warnings

warnings.filterwarnings("ignore", category=DeprecationWarning)
```

0.0.1 We'll just make some synthetic data to play with.

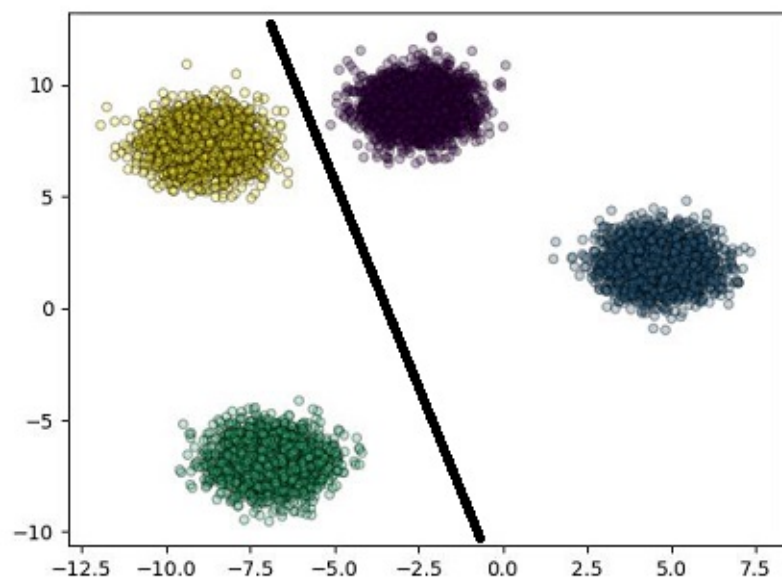
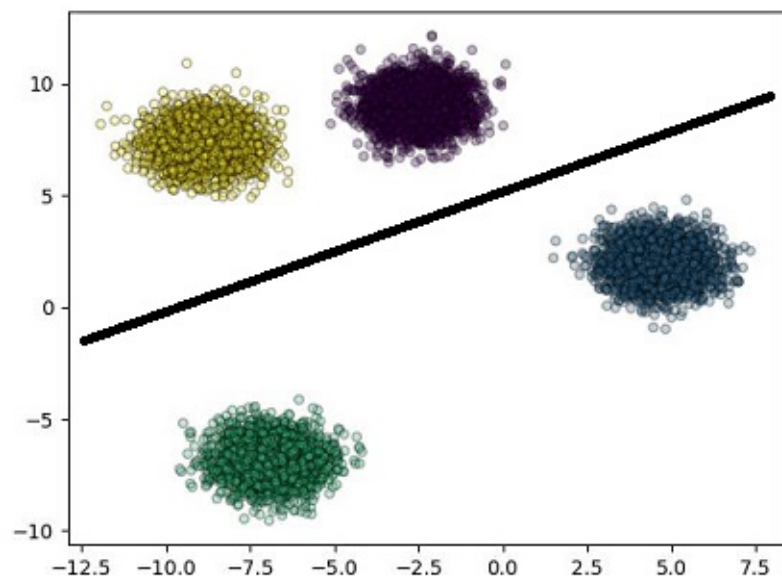
```
[2]: np.random.seed(42)
X, y = make_blobs(n_samples=10000, n_features=2, centers=4, cluster_std=.8)

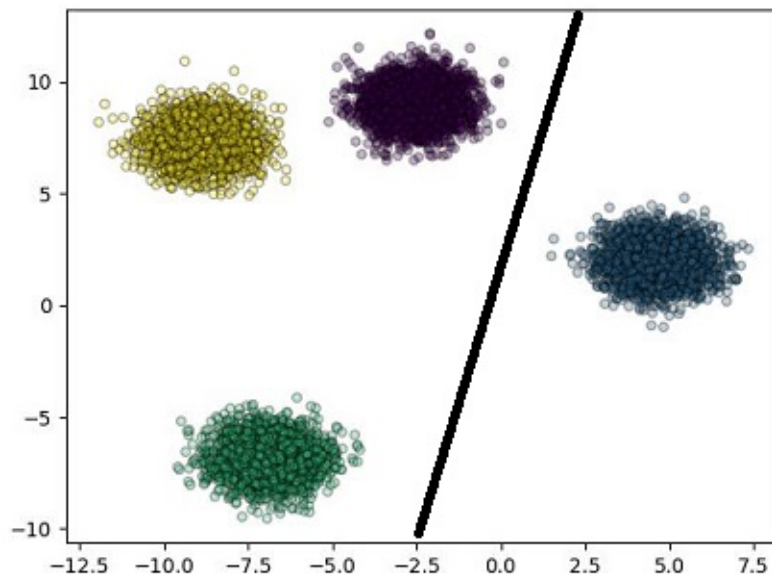
plt.scatter(X[:, 0], X[:, 1], marker='.', alpha=0.3, ec='k', s=80, c=y)
plt.savefig('blobs.jpg')
plt.show()
```



0.0.2 We can normally only find a single decision line using a binary classifier

We can separate one from the others or two against the other two, but we cannot isolate each class without additional steps.





So normally a binary classifier such as LogisticRegression will not be useful – without some added strategies. The two strategies we can employ for this are **One Vs. One** or **One Vs Rest** (or sometimes “Rest Vs all”). In fact, that we can separate one class from the other is the start of one such strategy (**One Vs. Rest**).

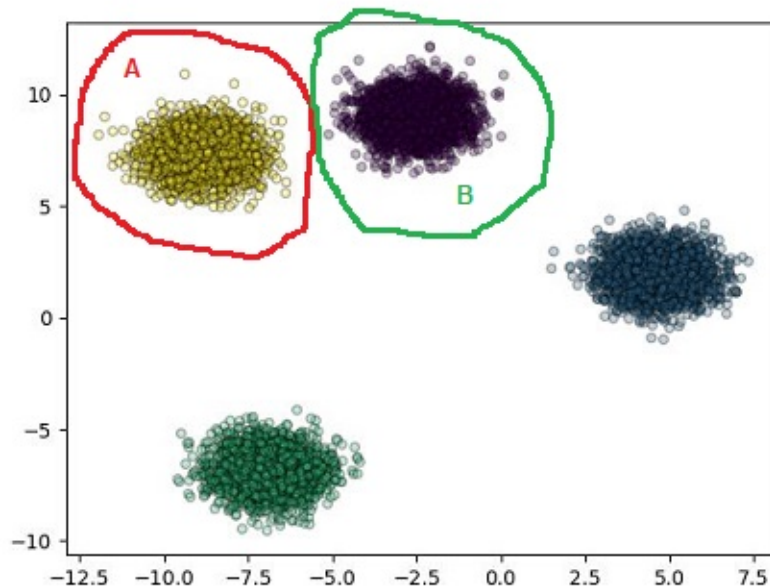
First, we can compare **One Vs. One** and **One Vs. Rest** (or All) strategies (thanks to Gemini).

Quick Comparison

Feature 	One-vs-One (OvO)	One-vs-All (OvA)
Number of Models	$N(N-1) \div 2$ (Quadratic)	N (Linear)
Data per Model	Small subset (Only the 2 target classes)	Entire dataset (1 target class vs. all others)
Training Speed	Slow for high number of classes	Fast (fewer total models to train)
Prediction Speed	Slower (requires voting across many models)	Faster (evaluates N models)
Imbalance Handling	Highly robust; balances the two paired classes	Vulnerable; easily skews toward the "rest" group

0.0.3 One Vs. One

In this strategy, we make a number of binary classifiers in which we decide between class **A** or **B** (or an example is either A, or not A, by implication then B). The other classes are, for the time being, basically ignored. For example,



We then do the same for each pair of classes, giving us:

- Class1 vs. Class2
- Class1 vs. Class3
- Class1 vs. Class4
- Class2 vs. Class3
- Class2 vs. Class4
- Class3 vs. Class4

This covers all the permutations of pairs of classes (reading both directions, you should find them all). We can also calculate the number of classifiers with this formula:

$$\frac{N(N-1)}{2}$$

N being the number of classes, so in our case:

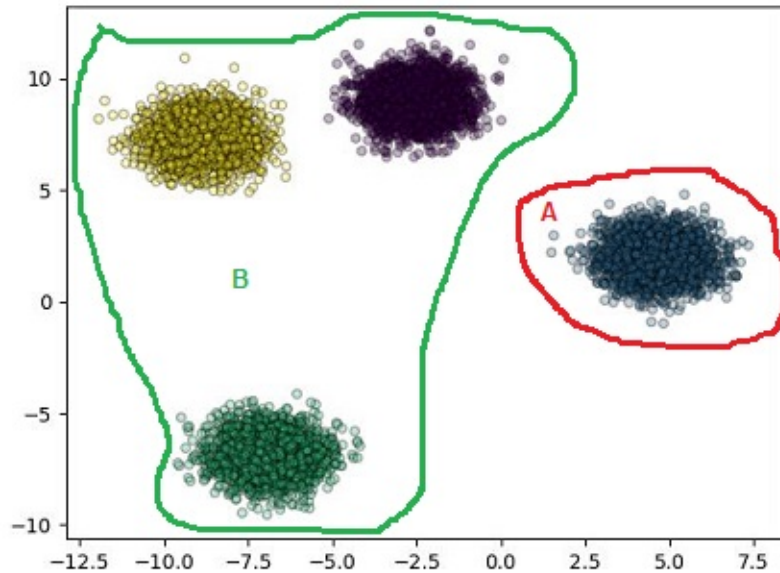
$$4(4-1)/2 = 12/2 = 6$$

With very large numbers of classes, *this strategy can become computationally expensive as the number of classifiers grow quadratically as the number of classes increases.*

This BTW, is an example of an **ensemble method** in which we break the problem space into multiple **weak classifiers** that, when combined, **can form a more powerful model**.

0.0.4 One Vs. Rest

In this strategy, we instead compare each class against all the other class, treated as one large class for the moment. For example:



We build a binary classifier for each class (**A**) against the other classes (**B**), or, more simply, we regard that each example is either a member of **A** or not a member of **B**. We then do the same with each class, i.e.:

- Class1 vs [Class2, Class3, Class4]
- Class2 vs [Class1, Class3, Class4]
- Class3 vs [Class1, Class2, Class4]
- Class4 vs [Class1, Class2, Class3]

Giving us 4 classifiers. In fact, the number of classifiers will always be the number of class, N , *making the growth of the number of classifiers linear*. Which in cases with a large number of classes, the resulting ensemble will be less computationally expensive than in the case of One-Vs-One.

0.0.5 Inferences

OK, this sounds all reasonable for training our model, but how do we apply such an ensemble on new data in which we want to infer the classes?

One Vs One

1. **The Evaluation:** Every model looks at the data point and picks a winner between its two assigned classes.
2. **The Tally:** The system aggregates all the votes into a scoreboard.
3. **The Winner:** The class that receives the highest number of total votes across all matchups wins the final prediction.

So, in our example, let's say we find that

- Model 1, Class1 Vs Class2, picks Class1
- Model 2, Class1 Vs Class3, picks Class1
- Model 3, Class1 Vs Class4, picks Class1
- Model 4, Class2 Vs Class3, picks Class3 (has to pick one)
- Model 5, Class2 Vs. Class4, picks Class2
- Model 6, Class3 Vs. Class4, picks Class3

Basically, by majority Class1 is the winner, so the model predicts Class1. (This is very typical of **ensemble models**, such also as with **Random Forest**. The **weak classifiers** all get a single vote, and the majority wins.)

One Vs. Rest

1. **The Evaluation:** Every classifier evaluates the new data point independently.
2. **The Winner:** The system compares all the scores and selects the class belonging to the model that returned the highest confidence score

This is actually the simplest evaluation. for example, we may find (just some made up values):

- Model 1 predicts Class1 with .80 confidence
- Model 2 predicts Class2 with .25 confidence (again, every model has to pick *something*)
- Model 3 predicts Class3 with .10 confidence
- Model 4 predicts Class4 with .10 confidence

In which case the model will finally predict Class1, the one with highest level of confidence. Notice that the confidence levels **do not** necessarily sum to 1: this is **not** a **probability distribution**, but rather *independent probabilities for each class against the rest*.

0.0.6 Let's try each!

First, we will split our data into train and test sets

```
[3]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.2,
↳ random_state=42)
```

And next we will scale the data, using the StandardScaler. *Notice this is actually the proper procedure: first split the data, and then apply scaling to each set as below.*

```
[4]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

We **fit_transform()** only on the **training set**, and then we only apply the statistics found in it on the **test set**. If we also fit the scaler on the test data, this can result in **data leakage** and *our model evaluation may not be reliable for the model's performance on future data*.

0.0.7 One Vs. One strategy

```
[5]: base_model = LogisticRegression()  
OvO_model = OneVsOneClassifier(estimator=base_model)  
OvO_model.fit(X_train_scaled, y_train)
```

```
[5]: OneVsOneClassifier(estimator=LogisticRegression())
```

```
[6]: y_hat = OvO_model.predict(X_test_scaled)  
print(classification_report(y_test, y_hat))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	500
1	1.00	1.00	1.00	509
2	1.00	1.00	1.00	475
3	1.00	1.00	1.00	516
accuracy			1.00	2000
macro avg	1.00	1.00	1.00	2000
weighted avg	1.00	1.00	1.00	2000

0.0.8 One Vs. All strategy

```
[7]: OvA_model = LogisticRegression(multi_class='ovr')  
OvA_model.fit(X_train_scaled, y_train)
```

C:\Users\karen\Anaconda3\envs\2u-data\Lib\site-packages\sklearn\linear_model_logistic.py:1281: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.8. Use OneVsRestClassifier(LogisticRegression(...)) instead. Leave it to its default value to avoid this warning.
warnings.warn(

```
[7]: LogisticRegression(multi_class='ovr')
```

```
[8]: y_hat = OvA_model.predict(X_test_scaled)  
print(classification_report(y_test, y_hat))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	500
1	1.00	1.00	1.00	509
2	1.00	1.00	1.00	475
3	1.00	1.00	1.00	516
accuracy			1.00	2000
macro avg	1.00	1.00	1.00	2000

weighted avg	1.00	1.00	1.00	2000
--------------	------	------	------	------

This is actually a very simple classification problem, with clear linear separations between classes (even if we need more than one decision line). However we can see that the strategies work and we can predict examples of each class accurately, which we can not do with a simple linear classifier.

[]: